UNIT-4

Lecture No.1

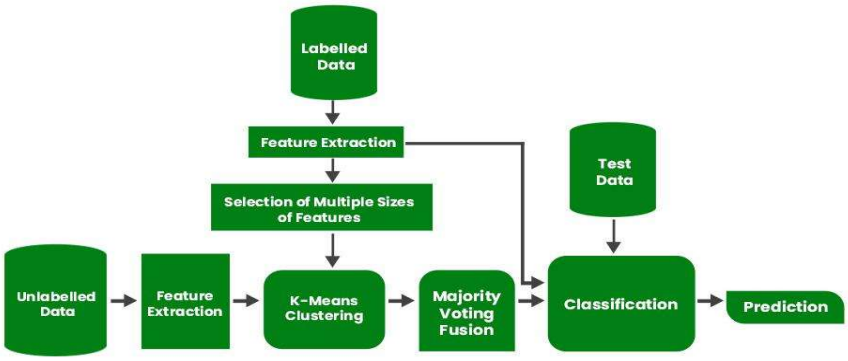
Semi-Supervised Learning in ML

Today's Machine Learning algorithms can be broadly classified into three categories, Supervised Learning, Unsupervised Learning, and Reinforcement Learning. Casting Reinforced Learning aside, the primary two categories of Machine Learning problems are Supervised and Unsupervised Learning. The basic difference between the two is that Supervised Learning datasets have an output label associated with each tuple while Unsupervised Learning datasets do not.

What is Semi-Supervised Learning?

Semi-supervised learning is a type of machine learning that falls in between supervised and unsupervised learning. It is a method that uses a small amount of labeled data and a large amount of unlabeled data to train a model. The goal of semi-supervised learning is to learn a function that can accurately predict the output variable based on the input variables, similar to supervised learning. However, unlike supervised learning, the algorithm is trained on a dataset that contains both labeled and unlabeled data.

Semi-supervised learning is particularly useful when there is a large amount of unlabeled data available, but it's too expensive or difficult to label all of it.



Semi-Supervised Learning Flow Chart

Intuitively, one may imagine the three types of learning algorithms as Supervised learning where a student is under the supervision of a teacher at both home and school, Unsupervised learning where a student has to figure out a concept himself and Semi-Supervised learning where a teacher teaches a few concepts in class and gives questions as homework which are based on similar concepts.

Examples of Semi-Supervised Learning:

- **Text classification**: In text classification, the goal is to classify a given text into one or more predefined categories. Semi-supervised learning can be used to train a text classification model using a small amount of labeled data and a large amount of unlabeled text data.
- **Image classification**: In image classification, the goal is to classify a given image into one or more predefined categories. Semi-supervised learning can be used to train an image classification model using a small amount of labeled data and a large amount of unlabeled image data.
- **Anomaly detection**: In anomaly detection, the goal is to detect patterns or observations that are unusual or different from the norm.

Applications of Semi-Supervised Learning:

1. **Speech Analysis:** Since labeling audio files is a very intensive task, Semi-Supervised learning is a very natural approach to solve this problem.
2. **Internet Content Classification:** Labeling each webpage is an impractical and unfeasible process and thus uses Semi-Supervised learning algorithms. Even the Google search algorithm uses a variant of Semi-Supervised learning to rank the relevance of a webpage for a given query.
3. **Protein Sequence Classification:** Since DNA strands are typically very large in size, the rise of Semi-Supervised learning has been imminent in this field.

Disadvantages of Semi-Supervised Learning:

The most basic disadvantage of any **Supervised Learning** algorithm is that the dataset has to be hand-labeled either by a Machine Learning Engineer or a Data Scientist. This is a very *costly process*, especially when dealing with large volumes of data.

The most basic disadvantage of any **Unsupervised Learning** is that its **application spectrum is limited**.

To counter these disadvantages, the concept of **Semi-Supervised Learning** was introduced. In this type of learning, the algorithm is trained upon a combination of labeled and unlabelled data. Typically, this combination will contain a very small amount of labeled data and a very large amount of unlabelled data. The basic procedure involved is that first, the programmer will cluster similar data using an unsupervised learning algorithm and then use the existing labeled data to label the rest of the unlabelled data. The typical use cases of such type of algorithm have a common property among them – The acquisition of unlabelled data is relatively cheap while labeling the said data is very expensive.

Lecture No.2

Reinforcement learning

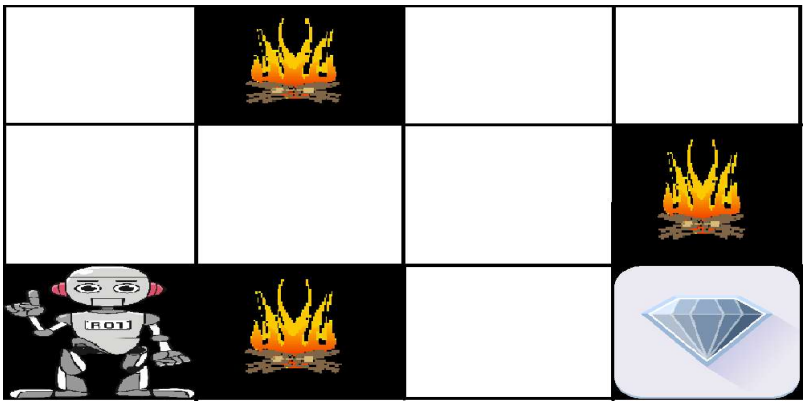
Reinforcement learning is an area of Machine Learning. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation. Reinforcement learning differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task. In the absence of a training dataset, it is bound to learn from its experience.

Reinforcement Learning (RL) is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward. In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input that we would find in supervised or unsupervised machine learning.

Reinforcement learning uses algorithms that learn from outcomes and decide which action to take next. After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect. It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

Reinforcement learning is an autonomous, self- teaching system that essentially learns by trial and error. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.

Example: The problem is as follows: We have an agent and a reward, with many hurdles in between. The agent is supposed to find the best possible path to reach the reward. The following problem explains the problem more easily.



The above image shows the robot, diamond, and fire. The goal of the robot is to get the reward that is the diamond and avoid the hurdles that are fired. The robot learns by trying all the possible paths and then choosing the path which gives him the reward with the least hurdles. Each right step will give the robot a reward and each wrong step will subtract the reward of the robot. The total reward will be calculated when it reaches the final reward that is the diamond.

Main points in Reinforcement learning –

- Input: The input should be an initial state from which the model will start
- Output: There are many possible outputs as there are a variety of solutions to a particular problem
- Training: The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.

Reinforcement learning	Supervised learning
Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input.	In Supervised learning, the decision is made on the initial input or the input given at the start.
In Reinforcement learning decision is dependent, So we give labels to sequences of dependent decisions	In supervised learning the decisions are independent of each other so labels are given to each decision.
Example: Chess game,text summarization	Example: Object recognition,spam detetction

Types of Reinforcement: There are two types of Reinforcement:

1. **Positive –**
   Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive

effect on behavior.
Advantages of reinforcement learning are:

- Maximizes Performance
- Sustain Change for a long period of time
- Too much Reinforcement can lead to an overload of states which can diminish the results

2. **Negative –**
Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.
Advantages of reinforcement learning:
   - Increases Behavior
   - Provide defiance to a minimum standard of performance
   - It Only provides enough to meet up the minimum behavior

Elements of Reinforcement Learning

Reinforcement learning elements are as follows:

1. Policy

2. Reward function

3. Value function

4. Model of the environment

Policy: Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
Reward function: Reward function is used to define a goal in a reinforcement learning problem. A reward function is a function that provides a numerical score based on the state of the environment
Value function: Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
Model of the environment: Models are used for planning.

The reinforcement learning problem model is an agent continuously interacting with an environment. The agent and the environment interact in a sequence of time steps. At each time step t, the agent receives the state of the environment and a scalar numerical reward for the previous action, and then the agent then selects an action.
Reinforcement learning is a technique for solving Markov decision problems.

Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem.

Various Practical applications of Reinforcement Learning –

- RL can be used in robotics for industrial automation.
- RL can be used in machine learning and data processing
- RL can be used to create training systems that provide custom instruction and materials according to the requirement of students.

Application of Reinforcement Learning:

1. Robotics: Robots with pre-programmed behavior are useful in structured environments, such as the assembly line of an automobile manufacturing plant, where the task is repetitive in nature.

2. A master chess player makes a move. The choice is informed both by planning, anticipating possible replies and counter replies.

3. An adaptive controller adjusts parameters of a petroleum refinery's operation in real time.

RL can be used in large environments in the following situations:

1. A model of the environment is known, but an analytic solution is not available;
2. Only a simulation model of the environment is given (the subject of simulation-based optimization)
3. The only way to collect information about the environment is to interact with it.

Advantages and Disadvantages of Reinforcement Learning:

Advantages of Reinforcement learning:

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.

2. The model can correct the errors that occurred during the training process.

3. In RL, training data is obtained via the direct interaction of the agent with the environment

Disadvantages of Reinforcement learning:

1. Reinforcement learning is not preferable to use for solving simple problems.

2. Reinforcement learning needs a lot of data and a lot of computation.

Lecture No.3

Markov Decision Process

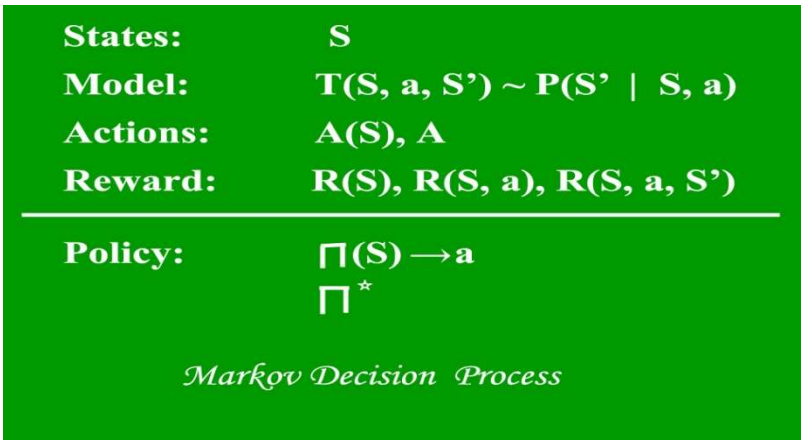
Reinforcement Learning :

Reinforcement Learning is a type of Machine Learning. It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance. Simple reward feedback is required for the agent to learn its behavior; this is known as the reinforcement signal.

There are many different algorithms that tackle this issue. As a matter of fact, Reinforcement Learning is defined by a specific type of problem, and all its solutions are classed as Reinforcement Learning algorithms. In the problem, an agent is supposed to decide the best action to select based on his current state. When this step is repeated, the problem is known as a **Markov Decision Process**.

A **Markov Decision Process (MDP)** model contains:

- A set of possible world states S.
- A set of Models.
- A set of possible actions A.
- A real-valued reward function R(s,a).
- A policy the solution of **Markov Decision Process**.



What is a State?

A **State** is a set of tokens that represent every state that the agent can be in.

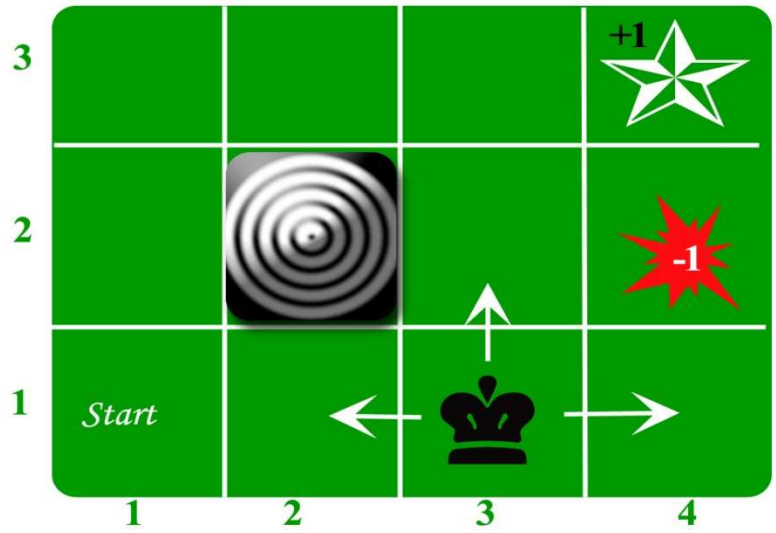
What is a Model?

A **Model** (sometimes called Transition Model) gives an action's effect in a state. In particular, T(S, a, S') defines a transition T where being in state S and taking an action 'a' takes us to state S' (S and S' may be the same). For stochastic actions (noisy, non-deterministic) we also define a probability P(S'|S,a) which represents the probability of reaching a state S' if action 'a' is taken in state S. Note Markov property states that the effects of an action taken in a state depend only on that state and not on the prior history.

What are Actions?
An **Action** A is a set of all possible actions. A(s) defines the set of actions that can be taken being in state S.

What is a Reward?
A **Reward** is a real-valued reward function. R(s) indicates the reward for simply being in the state S. R(S,a) indicates the reward for being in a state S and taking an action 'a'. R(S,a,S') indicates the reward for being in a state S, taking an action 'a' and ending up in a state S'.

What is a Policy?
A **Policy** is a solution to the Markov Decision Process. A policy is a mapping from S to a.
It indicates the action 'a' to be taken while in state S. Let us take the example of a grid world:



An agent lives in the grid. The above example is a 3*4 grid. The grid has a START state(grid no 1,1). The purpose of the agent is to wander around the grid to finally reach the Blue

Diamond (grid no 4,3). Under all circumstances, the agent should avoid the Fire grid (orange color, grid no 4,2). Also the grid no 2,2 is a blocked grid, it acts as a wall hence the agent cannot enter it.

The agent can take any one of these actions: **UP, DOWN, LEFT, RIGHT**
Walls block the agent path, i.e., if there is a wall in the direction the agent would have taken, the agent stays in the same place. So for example, if the agent says LEFT in the START grid he would stay put in the START grid.

First Aim: To find the shortest sequence getting from START to the Diamond.
Two such sequences can be found:

- **RIGHT RIGHT UP UPRIGHT**
- **UP UP RIGHT RIGHT RIGHT**

Let us take the second one (UP UP RIGHT RIGHT RIGHT) for the subsequent discussion. The move is now noisy. 80% of the time the intended action works correctly. 20% of the time the action agent takes causes it to move at right angles. For example, if the agent says UP the probability of going UP is 0.8 whereas the probability of going LEFT is 0.1, and the probability of going RIGHT is 0.1 (since LEFT and RIGHT are right angles to UP).

The agent receives rewards each time step:-

- Small reward each step (can be negative when can also be term as punishment, in the above example entering the Fire can have a reward of -1).
- Big rewards come at the end (good or bad).
- The goal is to Maximize the sum of rewards.

Lecture No.4

Bellman equations

The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman in the year 1953**, and hence it is called as a Bellman equation. It is associated with dynamic programming and used to calculate the values of a decision problem at a certain point by including the values of previous states.

According to the Bellman Equation**, long-term- reward in a** given action is equal to the reward from the current action combined with the expected reward from the future actions **taken at the following time.**

- Action performed by the agent is referred to as "a"
- State occurred by performing the action is "s."
- The reward/feedback obtained for each good and bad action is "R."
- A discount factor is Gamma "γ."

The Bellman equation can be written as:

$$V(s)=max_a(R(s,a)+\gamma V(s'))$$

Where,

V(s)= value calculated at a particular point.

R(s,a) = Reward at a particular state s by performing an action.

γ = Discount factor

V(s`) = The value at the previous state.

In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.

So now, using the Bellman equation, we will find value at each state of the given environment.

For 1st block:

V(s3) = max [R(s,a) + γV(s`)], here V(s')= 0 because there is no further state to move.

V(s3)= max[R(s,a)]=> V(s3)= max[1]=> **V(s3)= 1.**

For 2nd block:

V(s2) = max [R(s,a) + γV(s`)], here γ= 0.9(lets), V(s')= 1, and R(s, a)= 0, because there is no reward at this state.

V(s2)= max[0.9(1)]=> V(s)= max[0.9]=> **V(s2) =0.9**

For 3rd block:

V(s1) = max [R(s,a) + γV(s`)], here γ= 0.9(lets), V(s')= 0.9, and R(s, a)= 0, because there is no reward at this state also.

V(s1)= max[0.9(0.9)]=> V(s3)= max[0.81]=> **V(s1) =0.81**

For 4th block:

V(s5) = max [R(s,a) + γV(s`)], here γ= 0.9(lets), V(s')= 0.81, and R(s, a)= 0, because there is no reward at this state also.

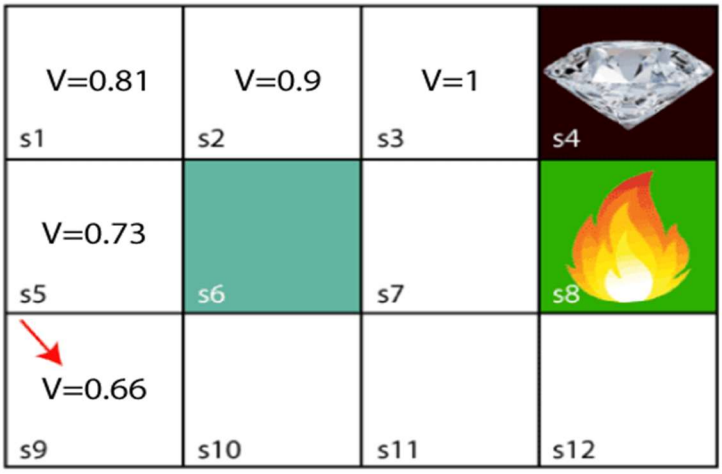
V(s5)= max[0.9(0.81)]=> V(s5)= max[0.81]=> **V(s5) =0.73**

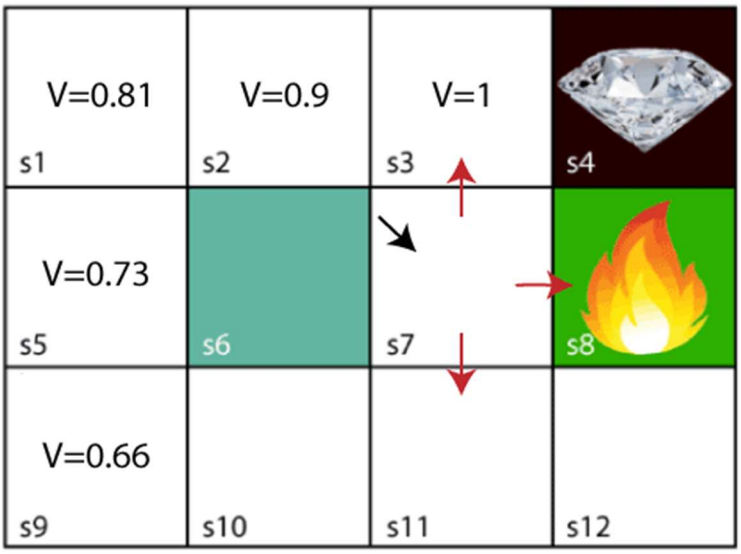
For 5th block:

V(s9) = max [R(s,a) + γV(s`)], here γ= 0.9(lets), V(s')= 0.73, and R(s, a)= 0, because there is no reward at this state also.

V(s9)= max[0.9(0.73)]=> V(s4)= max[0.81]=> **V(s4) =0.66**

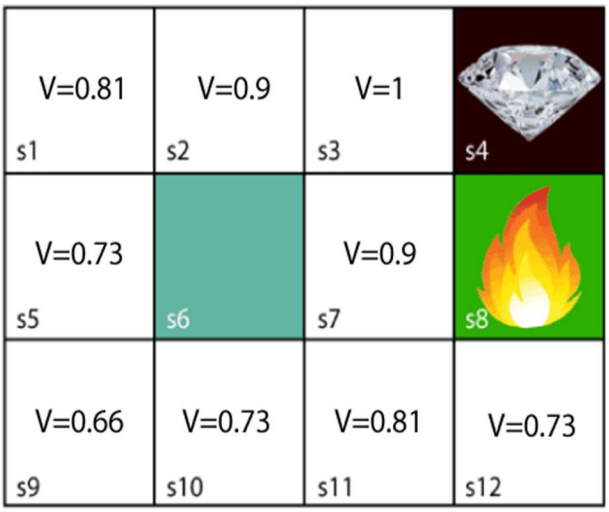
Consider the below image:





Now, we will move further to the 6th block, and here agent may change the route because it always tries to find the optimal path. So now, let's consider from the block next to the fire pit.

Now, the agent has three options to move; if he moves to the blue box, then he will feel a bump if he moves to the fire pit, then he will get the -1 reward. But here we are taking only positive rewards, so for this, he will move to upwards only. The complete block values will be calculated using this formula. Consider the below image:



Lecture No.5

Policy evaluation using Monte Carlo

Monte Carlo to solve the prediction/evaluation problem. Use Monte-Carlo to evaluate policy π only. This means we are looking for the state-value function v_π(s) given policy π. It would work for the action-value function q_π(s) also, but for simplicity, we concentrate on v_π(s) here.

Remember that the state-value is nothing more than the discounted cumulative reward by starting from state *s* following a policy π to the end state at the last timestep T:

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T$$

An **episode** is a sequence of all states from an initial state *s* to a terminal state. It is ***not*** a continuous and infinite sequence. Moreover, for each episode, we get a different return G_t. An example of an episode for a video game could be the sequence of all states, the frames, from start to Game Over.

Now, let's choose one of our states as an initial state *s_i*. We go through one complete episode by following the policy π and eventually reach the end state *s_e*.

We repeat this many, many times. On every iteration, the end state could be reached in a different way, depending on the stochastic policy π. This leads to a different return *G_t* for every episode. Averaging over those returns gives us a close representation of the real state-value for the initial state s_i.

If we do this process for all states, where we always pick a different state as the initial state, then we obtain a **good approximation** of the state-value for each state and therefore the state-value function of the environment model. This is the main idea behind Monte-Carlo.

Instead of using the Dynamic Programming and the Bellman Equation to calculate the exact value of the state s at timestep t by considering future rewards and values of the future states, we obtain an estimate of the true value function by running through many episodes, then observe what returns we got for each and average over all of them. No model is needed.

There are two different approaches to evaluate the policy by using Monte-Carlo. First-Visit and Every-Visit. For both, we keep a counter N(s) for each state s and we save the sum of the different returns from each episode for each state in S(s).

N(s) and S(s) accumulate values until all K episodes are completed. They are **not** set to zero after each episode.

Every-Visit Monte-Carlo Policy Evaluation

We iterate through K episodes. In each episode, every time we reach a state s we increment the counter N(s) for that state by one. It is possible that the same state is reached multiple times in the same episode. For all those visits the counter is increased.

$$N(s) \leftarrow N(s) + 1$$

On all those visits we also update the current S(s) value by adding the return G_t of the current episode k, starting from the state s to the terminal state. For each visit, G_t can be different.

$$S(s) \leftarrow S(s) + G_t$$

After having done that we update our current value for the state by doing the following:

$$V(s) \longleftarrow \frac{S(s)}{N(s)}$$

Which is just the average of all the returns of all episodes till now.

The algorithm looks like the following:

Initialize:

$\pi \leftarrow$ policy to be evaluated

$v \leftarrow$ an arbitrany state-value function

$S(s) \leftarrow 0$, for all $s \in S$

$N(s) \leftarrow 0$

Repeat for K episodes:

(a) generate an episode using π

(b) For each state in the episode:

$N(s) \leftarrow N(s) + 1$

$S(s) \leftarrow S(s) + G_t$

(c) $V(s) \leftarrow \frac{S(s)}{N(s)}$ for all $s \in S$

First-Visit Monte-Carlo Policy Evaluation

We do the same as for the Every-Time Monte-Carlo Policy Evaluation. The only difference here is that in the same episode we update N(s) and the S(s) only on the first visit of the state s. In our algorithm, we will check if s was visited for the first time in the episode and only then update them.

Initialize:

$\pi \leftarrow$ policy to be evaluated

$v \leftarrow$ an arbitrany state-value function

$S(s) \leftarrow 0$, for all $s \in S$

$N(s) \leftarrow 0$

Repeat for K episodes:

(a) generate an episode using π

(b) For each state in the episode:

if (first visit of s) {

$N(s) \leftarrow N(s) + 1$

$S(s) \leftarrow S(s) + G_t$

}

(c) $V(s) \leftarrow \frac{S(s)}{N(s)}$ for all $s \in S$

Incremental Monte-Carlo

Now, instead of going through all episodes first and update V(s) for all states in the end only, we update V(s) incrementally, after each episode. We don't need the visits S(s) anymore.

We calculate an error term between the real output of the episode G_t and our estimate V(s). Remember that G_t is always calculated from the current state onwards.And we update the V(s) in direction of the mean error:

$$G_t - V(s)$$

Have a look at the update after each episode:

$$\frac{1}{N(s)}$$

On each iteration, we move a bit closer to the sample we have seen and get closer and closer to the mean.

$$V(s) \leftarrow V(s) + \frac{1}{N(s)}(G_t - V(s))$$

And the algorithm:

Initialize:

$\pi \leftarrow$ policy to be evaluated

$v \leftarrow$ an arbitrany state-value function

$N(s) \leftarrow 0$

Repeat for K episodes:

(a) generate an episode using π

(b) For each state in the episode:

$N(s) \leftarrow N(s) + 1$

$V(s) \leftarrow V(s) + \frac{1}{N(s)}(G_t - V(s))$

Instead of using N(s) to consider all states, we will use a constant α. In this way, we just care about a fixed number of steps and allow forgetting some states.

This makes sense in more flexible contexts, in non-stationary problems. Which is actually almost the case in the real world. A robot does not need to consider every step it has ever taken to make a decision for the next step. Just some past steps are important. The downside is that by using the fixed step size α instead of the mean we under- or overcorrect our V(s).

For our update part, this means we just replace 1/N(s) with α.

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

It also makes our algorithm above a bit simpler because we don't need to keep track of N(s) anymore.

Problems of Montecarlo

- Working only for episodes, not for continuous or infinite problems
- We need to complete an episode first before we can update all values. That's bad e.g. for autonomous cars which if using Monte Carlo there would update the values of a state only after a crash, the end of the episode. That's a bit too late.

Lecture No.6
Policy iteration and Value iteration

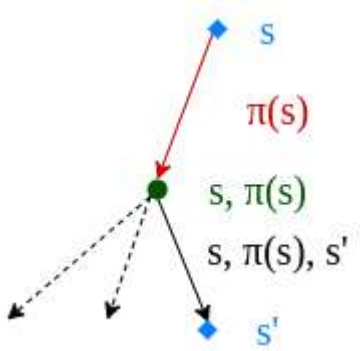
We can formulate a reinforcement learning problem via a Markov Decision Process (MDP). The essential elements of such a problem are the environment, state, reward, policy, and value.

A policy is a mapping from states to actions. Finding an optimal policy leads to generating the maximum reward. **Given an MDP environment, we can use dynamic programming algorithms to compute optimal policies, which lead to the highest possible sum of future rewards at each state.**

Dynamic programming algorithms work on the assumption that we have a perfect model of the environment's MDP. So, we're able to use a one-step look-ahead approach and compute rewards for all possible actions.

Policy Iteration

In policy iteration, we start by choosing an arbitrary policy. Then, we iteratively evaluate and improve the policy until convergence:



We evaluate a policy by calculating the state value function :

$$V(s) = \sum_{s',r'} p(s', r|s, \pi(s))[r + \gamma V(s')]$$

Then, we calculate the improved policy by using one-step look-ahead to replace the initial policy :

$$\pi(s) = arg\max_{a} \sum_{s',r'} p(s', r|s, a)[r + \gamma V(s')]$$

Here, is the reward generated by taking the action , is a discount factor for future rewards and is the transition probability.

In the beginning, we don't care about the initial policy being optimal or not. During the execution, we concentrate on improving it on every iteration by repeating policy evaluation and policy improvement steps. Using this algorithm we produce a chain of policies, where each policy is an improvement over the previous one:

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

We conduct policy evaluation and policy improvement steps until the policy doesn't improve anymore:

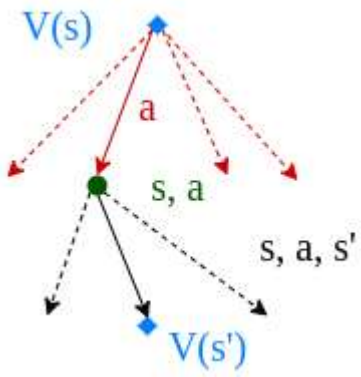
Algorithm 1: Policy Iteration Algorithm

Data: θ: a small number
Result: V: a value function s.t. $V \approx v_*$, π: a deterministic policy s.t. $\pi \approx \pi_*$
Function *PolicyIteration* **is**
  `/* Initialization */`
  Initialize $V(s)$ arbitrarily;
  Randomly initialize policy $\pi(s)$;
  `/* Policy Evaluation */`
  $\Delta \leftarrow 0$;
  while $\Delta < \theta$ **do**
    for *each* $s \in S$ **do**
      $v \leftarrow V(s)$;
      $V(s) \leftarrow \sum_{s',r'} p(s', r|s, \pi(s))[r + \gamma V(s')]$;
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$;
    end
  end
  `/* Policy Improvement */`
  policy-stable $\leftarrow$ *true*;
  for *each* $s \in S$ **do**
    old-action $\leftarrow \pi(s)$;
    $\pi(s) \leftarrow arg\max_a \sum_{s',r'} p(s', r|s, a)[r + \gamma V(s')]$;
    if *old-action* != $\pi(s)$ **then**
      policy-stable $\leftarrow$ *false*;
    end
  end
  if *policy − stable* **then**
    return $V \approx v_*$ and $\pi \approx \pi_*$;
  else
    go to Policy Evaluation;
  end
end

Since a finite MDP has a finite number of policies, the defined process is finite. In the end, converging an optimal policy V(s) and an optimal value function $\pi(s)$ is guaranteed.

Value Iteration

In value iteration, we compute the optimal state value function by iteratively updating the estimate :



We start with a random value function . At each step, we update it:

$$V(s) = \max_a \sum_{s',r'} p(s', r|s, a)[r + \gamma V(s')]$$

Hence, we look-ahead one step and go over all possible actions at each iteration to find the maximum:

Algorithm 2: Value Iteration Algorithm

Data: θ: a small number
Result: π: a deterministic policy s.t. $\pi \approx \pi_*$
Function *ValueIteration* **is**
  /* Initialization */
  Initialize $V(s)$ arbitrarily, except $V(terminal)$;
  $V(terminal) \leftarrow 0$;
  /* Loop until convergence */
  $\Delta \leftarrow 0$;
  while $\Delta < \theta$ **do**
    for *each* $s \in S$ **do**
      $v \leftarrow V(s)$;
      $V(s) \leftarrow \max_a \sum_{s',r'} p(s', r|s, a)[r + \gamma V(s')]$;
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$;
    end
  end
  /* Return optimal policy */
  return π s.t. $\pi(s) = arg\max_a \sum_{s',r'} p(s', r|s, a)[r + \gamma V(s')]$;
end

The update step is very similar to the update step in the policy iteration algorithm. The only difference is that we take the maximum over all possible actions in the value iteration algorithm.

Instead of evaluating and then improving, the value iteration algorithm updates the state value function in a single step. This is possible by calculating all possible rewards by looking ahead.The value iteration algorithm is guaranteed to converge to the optimal values.

Policy Iteration vs. Value Iteration

Policy iteration and value iteration are both dynamic programming algorithms that find an optimal policy in a reinforcement learning environment. They both employ variations of Bellman updates and exploit one-step look-ahead:

Policy Iteration	Value Iteration
Starts with a random policy	Starts with a random value function
Algorithm is more complex	Algorithm is simpler
Guaranteed to converge	Guaranteed to converge
Cheaper to compute	More expensive to compute
Requires few iterations to converge	Requires more iterations to converge
Faster	Slower

In policy iteration, we start with a fixed policy. Conversely, in value iteration, we begin by selecting the value function. Then, in both algorithms, we iteratively improve until we reach convergence.

The policy iteration algorithm updates the policy. The value iteration algorithm iterates over the value function instead. Still, both algorithms implicitly update the policy and state value function in each iteration.

In each iteration, the policy iteration function goes through two phases. One phase evaluates the policy, and the other one improves it. The value iteration function covers these two phases by taking a maximum over the utility function for all possible actions.

The value iteration algorithm is straightforward. It combines two phases of the policy iteration into a single update operation. However, the value iteration function runs through all possible actions at once to find the maximum action value. Subsequently, the value iteration algorithm is computationally heavier.

Both algorithms are guaranteed to converge to an optimal policy in the end. Yet, the policy iteration algorithm converges within fewer iterations. As a result, the policy iteration is reported to conclude faster than the value iteration algorithm.

Lecture No.7
Q-Learning State- Action-Reward-State-Action (SARSA)

SARSA algorithm is a slight variation of the popular Q-Learning algorithm. For a learning agent in any Reinforcement Learning algorithm it's policy can be of two types:-

1. **On Policy:** In this, the learning agent learns the value function according to the current action derived from the policy currently being used.

2. **Off Policy:** In this, the learning agent learns the value function according to the action derived from another policy.

Q-Learning technique is an **Off Policy** technique and uses the greedy approach to learn the Q-value. SARSA technique, on the other hand, is an **On Policy** and uses the action performed by the current policy to learn the Q-value. This difference is visible in the difference of the update statements for each technique:-

1. Q-Learning:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

2. SARSA:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

Here, the update equation for SARSA depends on the current state, current action, reward obtained, next state and next action. This observation lead to the naming of the learning technique as SARSA stands for **State Action Reward State Action** which symbolizes the tuple (s, a, r, s', a').

SARSA (State-Action-Reward-State-Action) is a reinforcement learning algorithm that is used to learn a policy for an agent interacting with an environment. It is a type of on-policy algorithm, which means that it learns the value function and the policy based on the actions that are actually taken by the agent.

The SARSA algorithm works by maintaining a table of action-value estimates Q(s, a), where s is the state and a is the action taken by the agent in that state. The table is initialized to some arbitrary values, and the agent uses an epsilon-greedy policy to select actions.

Here are the steps involved in the SARSA algorithm:
Initialize the action-value estimates Q(s, a) to some arbitrary values.

1. Set the initial state s.
2. Choose the initial action a using an epsilon-greedy policy based on the current Q values.
3. Take the action a and observe the reward r and the next state s'.
4. Choose the next action a' using an epsilon-greedy policy based on the updated Q values.
5. Update the action-value estimate for the current state-action pair using the SARSA update rule:
6. Q(s, a) = Q(s, a) + alpha * (r + gamma * Q(s', a') – Q(s, a))

where alpha is the learning rate, gamma is the discount factor, and r + gamma * Q(s', a') is the estimated return for the next state-action pair.

7. Set the current state s to the next state s', and the current action a to the next action a'.

Repeat steps 4-7 until the episode ends.

The SARSA algorithm learns a policy that balances exploration and exploitation, and can be used in a variety of applications, including robotics, game playing, and decision making. However, it is important to note that the convergence of the SARSA algorithm can be slow, especially in large state spaces, and there are other reinforcement learning algorithms that may be more effective in certain situations.

Lecture No.8
Model-based Reinforcement Learning.

What is Model-Based Machine Learning?

Hundreds of learning algorithms have been developed in the field of machine learning. Scientists typically select from among these algorithms to answer specific issues. Their options are frequently restricted by their expertise with these systems. In this classical/traditional machine learning framework, scientists are forced to make some assumptions to employ an existing algorithm.

- **The model-based learning in machine learning is a technique that tries to generate a custom solution for each new challenge**

MBML's purpose is to **offer a single development framework that facilitates the building of a diverse variety of custom models**. This paradigm evolved as a result of a significant confluence of three main ideas:

- **Factor graphs**
- **Bayesian perspective,**
- **Probabilistic Programming**

The essential principle is that in the form of a model, all assumptions about the issue domain are made clear. Model-based deep learning is just a collection of assumptions stated in a graphical manner.

Factor Graphs

The usage of PGM- Probabilistic Graphical Models, particularly factor graphs, is the pillar of MBML. A PGM is a graph-based diagrammatic representation of the joint probability distribution across all random variables in a model.

They are a form of PGM with round nodes and square nodes representing variable probability distributions (factors), and vertices expressing conditional relationships between nodes. They offer a broad framework for simulating the combined dispersion of a set of random variables.

In factor graphs, we consider implicit parameters as random variables and discover their probability distributions throughout the network using Bayesian inference techniques. Inference/learning is just the product of factors across a subset of the graph's variables. This makes it simple to develop local message forwarding algorithms.

Bayesian Methods

The first essential concept allowing this new machine learning architecture is Bayesian inference/learning. Latent/hidden parameters are represented in MBML as random variables with probability distributions. This provides for a consistent and rational approach to quantifying uncertainty in model parameters. Again when the observed variables in the model are locked to their values, the Bayes' theorem is used to update the previously assumed probability distributions.

In contrast, the classical ML framework assigns model parameters to average values derived by maximizing an objective function. Bayesian inference on big models with millions of variables is accomplished similarly, but in a more complicated way, employing the Bayes' theorem. This is because Bayes' theory is an accurate inference approach that is intractable when applied to huge datasets. The rise in the processing capacity of computers over the last decade has enabled the research and innovation of algorithms that can scale to enormous data sets.

Probabilistic Programming

Probabilistic programming (PP) is a breakthrough in computer science in which programming languages are now created to compute with uncertainty in addition to logic. Current programming languages can already handle random variables, variable restrictions, and inference packages. You may now express a model-based reinforcement learning of your problem concisely with a few lines of code using a PP language. So an inference engine is invoked to produce inference procedures to solve the problem automatically.

Model-Based ML Developmental Stages

It consists of three rules-based models in machine learning:

- **Describe the Model**: Using factor graphs, describe the process that created the data.
- **Condition on Reported Data**: Make the observed variables equal to their known values.
- **Backward reasoning** is used to update the prior distribution across the latent constructs or parameters. Estimate the Bayesian probability distributions of latent constructs based on observable variables.